3F8: Inference
Short Lab Report

Jaehyung Choi

March 2, 2026

Abstract

This report applies logistic regression to a 2D binary classification task, fitting parameters via gradient ascent of the log-likelihood. We compare a linear model (test LL ≈ -0.629) against non-linear models that use Gaussian RBF feature expansions with widths $l \in \{0.01, 0.1, 1\}$. With learning rates chosen according to the spectral radius of each feature matrix, $l = 1$ gives the best test performance (LL ≈ -0.241), while $l = 0.01$ fails to converge and $l = 0.1$ offers a good intermediate result (LL ≈ -0.419).

1 Introduction

Logistic regression models the probability of a binary label $y \in \{0, 1\}$ given input $\mathbf{x} \in \mathbb{R}^2$ as $p(y = 1 \mid \mathbf{x}) = \sigma(\beta^\top \tilde{\mathbf{x}})$, where σ is the logistic sigmoid and $\tilde{\mathbf{x}}$ is $\mathbf{x}$ with a prepended 1 (bias). Parameters β are found by gradient ascent on the log-likelihood.

A linear model can only produce straight decision boundaries, so when the data is not linearly separable we might do better with richer features. Here we replace the raw 2D inputs with Gaussian radial basis function (RBF) features centred on the training points and compare the effect of three kernel widths $l \in \{0.01, 0.1, 1\}$ on test log-likelihood and classification accuracy.

2 Exercise a)

The log-likelihood for logistic regression over N data points is

$$\mathcal{L}(\beta) = \sum_{n=1}^{N} \left[y^{(n)} \log \sigma\Big(\beta^\top \tilde{x}^{(n)}\Big) + (1 - y^{(n)}) \log\Big(1 - \sigma\Big(\beta^\top \tilde{x}^{(n)}\Big)\Big) \right].$$

To differentiate, we use the identities $\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z)$ and $\frac{d}{dz} \log(1 - \sigma(z)) = -\sigma(z)$, which give

$$\frac{\partial \log \sigma(\beta^\top \tilde{x}^{(n)})}{\partial \beta} = \big(1 - \sigma(\beta^\top \tilde{x}^{(n)})\big)\tilde{x}^{(n)}, \quad \frac{\partial \log(1 - \sigma(\beta^\top \tilde{x}^{(n)}))}{\partial \beta} = -\sigma(\beta^\top \tilde{x}^{(n)})\tilde{x}^{(n)}.$$

Multiplying each term by $y^{(n)}$ or $(1 - y^{(n)})$ and summing:

$$\frac{\partial \mathcal{L}(\beta)}{\partial \beta} = \sum_{n=1}^{N} \big(y^{(n)} - \sigma(\beta^\top \tilde{x}^{(n)})\big)\tilde{x}^{(n)} = \tilde{\mathbf{X}}^\top \big(\mathbf{y} - \sigma(\tilde{\mathbf{X}}\beta)\big),$$

where $\tilde{\mathbf{X}}$ is the $N \times (D{+}1)$ design matrix with rows $\tilde{x}^{(n)\top}$ and $\sigma(\cdot)$ acts element-wise.

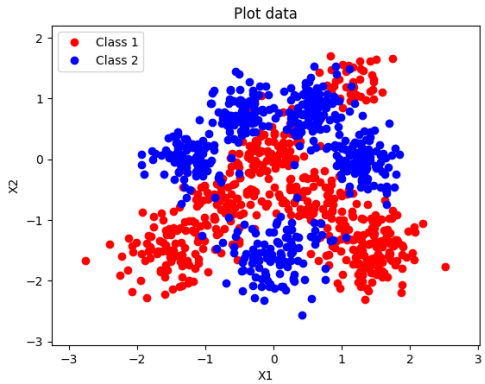

Figure 1: Visualisation of the data.

3 Exercise b)

Vectorised pseudocode for gradient ascent:

```
Function estimate_parameters(X, y, eta, T):

   Input:  N x D feature matrix X, N-vector y,
           learning rate eta, number of steps T
   Output: (D+1)-vector beta

   X_tilde <- prepend column of 1s to X     # N x (D+1)
   beta    <- random initialisation

   for i = 1 to T:
       s    <- sigmoid(X_tilde @ beta)       # N-vector
       grad <- X_tilde^T @ (y - s)           # (D+1)-vector
       beta <- beta + eta * grad

   return beta
```

We set $\eta = 1/N$ (here 1/800). Dividing by N keeps the effective step size independent of dataset size. To check convergence we plot the average log-likelihood at each step for both training and test sets.

4 Exercise c)

Figure 1 shows the dataset. The two classes (red and blue) overlap in the middle of the input space, so a linear boundary will inevitably misclassify some points. This motivates trying non-linear features later.

5 Exercise d)

We split the 1000 points into 800 training and 200 test. The gradient update in Python is:

```
sigmoid_value = predict(X_tilde_train, w)
w = w + alpha * np.dot((y_train - sigmoid_value), X_tilde_train)
```

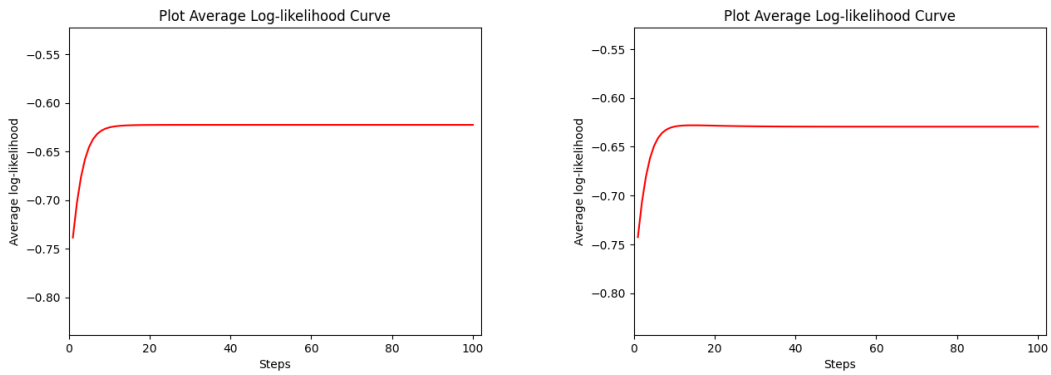

Figure 2: Learning curves for the linear classifier: training (left) and test (right) average log-likelihood.

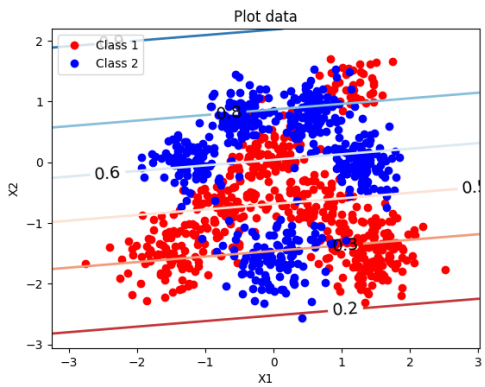

Figure 3: Contour plot of the linear classifier's predictive probabilities.

With $\eta = 1/800$ and 100 steps, the training and test log-likelihoods (Figure 2) both rise sharply in the first $\sim$20 steps and flatten out by step $\sim$50. The two curves stay close together throughout, so overfitting is not an issue here.

The contour plot (Figure 3) confirms a straight decision boundary at $p = 0.5$, cutting through the overlap region.

6 Exercise e)

Table 1 gives the converged log-likelihoods. The gap between train (-0.6225) and test (-0.6294) is small, consistent with the learning curves in Figure 2.

The confusion matrix (Table 2) shows the model gets 68% of Class 0 and 76% of Class 1 correct. The relatively high false-positive rate (32%) reflects the overlap visible in Figure 1 — points from both classes sit on the wrong side of the linear boundary.

7 Exercise f)

We expand the inputs using Gaussian RBF features centred on the 800 training points, with widths $l \in \{0.01, 0.1, 1\}$. Learning rates were set to $\eta = 0.01/800$, $0.1/800$, $0.04/800$ respectively, with 1000, 1000, and 5000 gradient steps. The contour plots are shown in Figure 4.

Avg. Train ll	Avg. Test ll
-0.6225	-0.6294

Table 1: Average log-likelihoods for the linear classifier.

		$\hat{y}$	
		0	1
y	0	0.6818	0.3182
	1	0.2444	0.7556

Table 2: Confusion matrix on the test set.

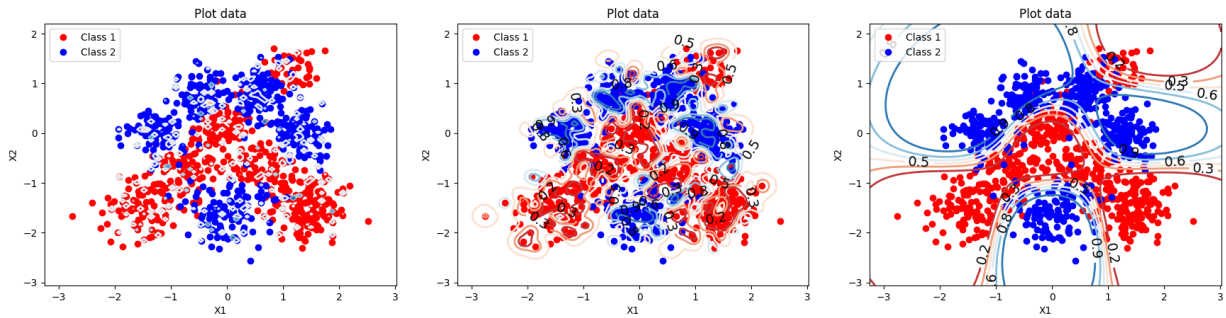

Figure 4: Contours of the class predictive probabilities for $l = 0.01$ (left), $l = 0.1$ (middle), $l = 1$ (right).

For $l = 0.01$ (left), the basis functions are so narrow that the feature matrix is nearly sparse — each training point only activates its own basis function. The gradients end up very small and the model barely moves from its initialisation in 1000 steps; the contour plot shows no meaningful decision boundary, and the learning curves (Figure 5, top row) remain nearly flat throughout. For $l = 0.1$ (middle), the boundary is smooth and curved, fitting the shape of the class overlap better than a straight line, and both log-likelihood curves converge steadily. For $l = 1$ (right), the broad basis functions overlap heavily, so the boundary is smoother but still noticeably non-linear; the training LL converges to a higher value than the test LL, hinting at mild overfitting.

8 Exercise g)

Tables 3–5 show the final log-likelihoods. The $l = 0.01$ model has not converged: both its train (-0.796) and test (-0.700) log-likelihoods are worse than the linear model, consistent with the degenerate contour plot. The $l = 0.1$ model gives a test LL of -0.419, a clear improvement over the linear baseline. The $l = 1$ model achieves the best performance overall: train LL -0.222 and test LL -0.241, with a train–test gap of only 0.019, indicating the model generalises well without overfitting.

The confusion matrices (Tables 6–8) tell a similar story. For $l = 0.01$, the model predicts Class 1 for nearly everything (FP= 0.96), confirming it has not learned anything useful. For $l = 0.1$, both classes are classified at about 82% accuracy, a clear step up from the linear model (68%/76%). For $l = 1$, Class 1 accuracy rises to 98% while Class 0 accuracy sits at 85% — the model is somewhat biased toward predicting Class 1, though both classes are classified more accurately than the linear model. Overall, $l = 1$ with the corrected learning rate gives the best test log-likelihood and accuracy among all models tested.

Avg. Train ll	Avg. Test ll
-0.7957	-0.7003

Table 3: Results for $l = 0.01$

Avg. Train ll	Avg. Test ll
-0.4597	-0.4187

Table 4: Results for $l = 0.1$

Avg. Train ll	Avg. Test ll
-0.2217	-0.2408

Table 5: Results for $l = 1$

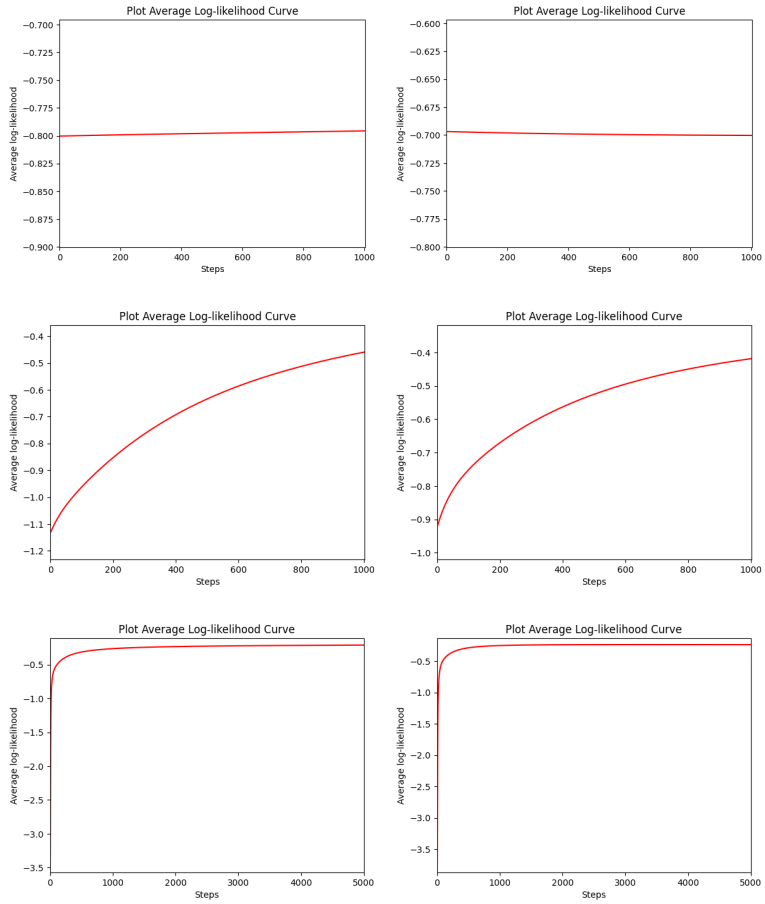

Figure 5: Training (left) and test (right) average log-likelihood curves for RBF classifiers with $l = 0.01$ (top), $l = 0.1$ (middle), $l = 1$ (bottom).

		$\hat{y}$	
		0	1
y	0	0.0364	0.9636
	1	0.1333	0.8667

Table 6: Conf. matrix $l = 0.01$.

		$\hat{y}$	
		0	1
y	0	0.8182	0.1818
	1	0.1778	0.8222

Table 7: Conf. matrix $l = 0.1$.

		$\hat{y}$	
		0	1
y	0	0.8545	0.1455
	1	0.0222	0.9778

Table 8: Conf. matrix $l = 1$.

9 Conclusions

The linear classifier converges quickly and gives a train/test LL of $-0.623/-0.629$ with no sign of overfitting, but misclassifies a substantial fraction of both classes due to the non-linear overlap in the data.

RBF feature expansion can improve on this, but the kernel width l and a suitable learning rate both matter. With $l = 0.01$ the features are too sparse for gradient ascent to make progress. With $l = 0.1$ we get a test LL of -0.419 and balanced accuracy ($\sim$82% on both classes). With $l = 1$, using a learning rate informed by the spectral radius of the feature matrix ($\eta = 0.04/800$, 5000 steps), the model achieves the best test LL (-0.241) and generalises well (train–test gap of only 0.019). This highlights that an inappropriate learning rate can mask a model's true capacity: the original $\eta = 0.5/800$ caused oscillations and poor convergence for $l = 1$, while the corrected rate reveals it as the strongest model. In practice, both l and η should be chosen carefully.